

Tecnicatura Universitaria en Programación

Metodología de Sistemas II

Objetivos

- Reconocer patrones de diseño de desarrollo de software.
- Implementar buenas prácticas en el desarrollo de software.
- Aplicar mejora continua durante todo el ciclo de desarrollo de software.
- Utilizar herramientas de verificación y validación en el desarrollo de software

Contenidos Minimos

- Introducción a los patrones de diseño y desarrollo de software
- Buenas prácticas en el proceso de implementación de software
- Técnicas de optimización del ciclo de desarrollo de software
- Herramientas de verificación y validación en el desarrollo de software
- Herramientas de repositorios de software

Introducción a los patrones de diseño y desarrollo de software

- Toda la información [aquí](#)
- Y mas [aquí](#)
- Basado en el libro [Head First Design Patterns](#) pero en C#
- Aquí el [libro](#)

Buenas prácticas en el proceso de implementación de software

Las buenas prácticas en la implementación de software incluyen:

- La planificación detallada con gestión de riesgos.
- El uso de metodologías ágiles para entregas incrementales.
- La automatización de pruebas (CI/CD).
- El control de versiones y documentación sólida.

Es crucial fomentar la comunicación constante del equipo, realizar revisiones de código y definir requerimientos claros para asegurar la calidad y reducir el retrabajo.

Buenas Prácticas Clave en la Implementación de Software

Planificación y Gestión de Proyecto

- Definir requerimientos claros: Establecer historias de usuario y priorizarlas antes de codificar.
- Planificación estructurada: Crear un plan de despliegue con plazos, recursos y estrategias de mitigación de riesgos.
- Entregas incrementales: Dividir el desarrollo en fases pequeñas y funcionales en lugar de esperar al final, facilitando la detección temprana de errores.
- Estimaciones realistas: Evitar presiones por plazos irreales que comprometan la calidad

Proceso de Desarrollo y Calidad

- Control de versiones: Utilizar herramientas para gestionar cambios en el código base.
- Revisiones de código (Code Reviews): Examinar el trabajo entre pares para mejorar la calidad y compartir conocimiento.
- Integración y Entrega Continua (CI/CD): Automatizar pruebas y despliegues para garantizar seguridad y rapidez en el paso a producción.
- Documentación adecuada: Mantener actualizada la documentación para facilitar el mantenimiento futuro.

Colaboración y Comunicación

- Equipos alineados: Utilizar herramientas como Jira, Slack o Microsoft Teams para la coordinación diaria.
- Programación en parejas: Considerar esta técnica para aumentar la calidad del código, especialmente en entornos de desarrollo ágil (XP).

Lanzamiento y Post-implementación

- Pruebas automáticas: Realizar pruebas continuas en cada iteración.
- Planes de reversión (Rollback): Desarrollar procedimientos claros para volver a una versión anterior si surgen problemas.

Implementar estas prácticas, basadas en la experiencia técnica y metodologías ágiles, reduce riesgos, costes y mejora la mantenibilidad del software a largo plazo.

Técnicas de optimización del ciclo de desarrollo de software

Optimizar el ciclo de vida del desarrollo de software (SDLC) implica

- Adoptar metodologías ágiles (Scrum, Kanban)
- Automatizar pruebas y despliegues (CI/CD)
- Utilizar Inteligencia Artificial para reducir tareas repetitivas y mejorar la calidad mediante la detección temprana de errores.

Estas técnicas aceleran la entrega, reducen costes y garantizan un software de mayor calidad.

Técnicas Clave de Optimización

- Metodologías Ágiles (Agile): Implementar Scrum o Kanban para mejorar la flexibilidad, permitir entregas iterativas y adaptar los cambios rápidamente.
- Integración y Entrega Continua (CI/CD): Automatizar la integración y despliegue de código con herramientas como Jenkins o Travis CI para reducir el riesgo de errores y acelerar el tiempo de comercialización.
- Automatización de Pruebas (Testing): Realizar pruebas exhaustivas automáticas (unitarias, integración, aceptación) desde etapas tempranas para asegurar la calidad y evitar fallos mayores.
- Uso de Inteligencia Artificial (IA): Emplear IA para la automatización de tareas repetitivas, análisis de datos, detección de patrones y mejora en la predicción de riesgos.
- Gestión del Ciclo de Vida (SDLC): Definir claramente los requisitos, utilizar diagramas para el diseño y realizar monitoreo constante tras la implementación.
- Automatización y Monitorización: Establecer sistemas de monitoreo para detectar problemas rápidamente y realizar actualizaciones periódicas para mantener la seguridad y funcionalidad.

Beneficios de la Optimización:

- Mayor eficiencia y calidad: Mejora del rendimiento del software y reducción de cuellos de botella.
- Entregas más rápidas: Reducción de los tiempos de desarrollo.
- Reducción de costos: Menos tiempo dedicado a la corrección de errores en etapas tardías.

Herramientas de verificación y validación en el desarrollo de software:

Las herramientas de verificación y validación (V&V) de software son esenciales para asegurar la calidad, funcionalidad y seguridad de una aplicación, detectando errores tempranamente.

Incluyen soluciones para pruebas automatizadas (Selenium, Cypress), análisis estático de código (SonarQube), gestión de pruebas (TestRail) y pruebas de carga (Apache JMeter), garantizando que el producto cumple los requisitos técnicos y del usuario.

Herramientas Principales de V&V

Verificación (Análisis Estático - Revisión sin ejecución):

- [SonarQube](#): Inspección continua de código para detectar bugs, vulnerabilidades y deuda técnica.
- [Kiuwan](#): Análisis estático para calidad y seguridad de código.
- [Nmap](#): Escaneo de seguridad y vulnerabilidades.

Validación (Pruebas Dinámicas - Ejecución del software):

- [Selenium](#): Automatización de pruebas funcionales en navegadores web.
- [Playwright](#): Pruebas automatizadas de extremo a extremo (E2E).
- [Cypress](#): Solución moderna de testing para aplicaciones web.
- [Katalon Studio](#): Plataforma integral para automatización y validación.
- [Postman](#): Pruebas y validación de APIs.
- [Apache JMeter](#): Pruebas de rendimiento y carga.

Gestión y Control de Calidad (QA):

- [TestRail](#): Control de casos de prueba y trazabilidad.
- [Allure TestOps](#): Reportes de pruebas de calidad.

Diferencia Clave

- Verificación: ¿Estamos construyendo el producto correctamente? (Revisar si el código cumple las especificaciones, usando herramientas estáticas).
- Validación: ¿Estamos construyendo el producto correcto? (Probar si la aplicación cumple las necesidades del usuario, usando pruebas dinámicas).

Herramientas de repositorios de software:

Las principales herramientas de repositorios de software son plataformas basadas en la nube que utilizan el sistema de control de versiones Git para almacenar, gestionar y colaborar en el código fuente. Las opciones líderes son GitHub (mayor comunidad), GitLab (fuerte en CI/CD) y Bitbucket (integrado con Jira).

Herramientas Principales de Repositorios (Git-based)

- [GitHub](#): La plataforma más grande, ideal para proyectos open source y colaboraciones masivas, con integraciones de IA (Copilot) y DevOps.
- [GitLab](#): Destaca por sus funcionalidades avanzadas de CI/CD (integración y despliegue continuos) y DevSecOps integrado.
- [Bitbucket](#): Enfocado a equipos empresariales, con fuerte integración dentro de la suite Atlassian (Jira, Trello) y repositorios privados.
- [Harness Code Repository](#): Una alternativa moderna para gestión de código y flujos DevOps.

Repositorios de Paquetes y Bibliotecas

Estas herramientas gestionan dependencias y artefactos compilados:

- [Maven](#): Para proyectos Java.
- [NPM](#): Para paquetes de Node.js.
- [NuGet](#): Para .NET.
- [Hackage](#): Para Haskell.

Repositorios Institucionales/Gestión

- [DSpace](#), [EPrints](#), [Fedora](#): Utilizados comúnmente en ámbitos académicos para almacenar activos digitales y publicaciones.

Estas herramientas permiten

- El control de versiones
- Revisión de código
- Seguimiento de incidencias
- Automatización del flujo de trabajo de desarrollo.